

C# DO JEITO CERTO

AULA 7 ARQUITETURA E DESIGN COM C#

Por que fazemos o que fazemos?

O desafio é reduzir o **custo/risco** da
mudança ao longo do tempo.



CAPÍTULO 1

ACOPLAMENTO

Golden Circle aplicado



Por quê (centro)

Reduzir o custo/risco da mudança

Como (meio)

Combatendo o acoplamento

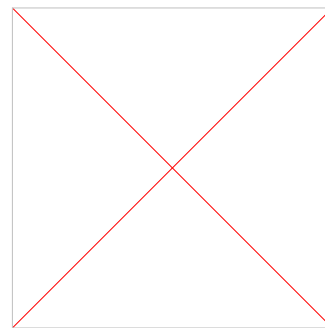
O quê (externo)

Clean, Hexagonal, VSA, Padrões de Design

*"Quando alguém diz 'estou usando Clean Architecture', está descrevendo um **o quê**. A pergunta interessante é o **porquê**. Sem porquê, o quê vira religião."*

Acoplamento é a dívida que mais cobra juros

- Acoplamento determina
o que muda junto
- Acoplamento determina
o que quebra junto
- Acoplamento determina
o que precisa ser entendido junto



*“Toda vez que você ouve ‘esse projeto é difícil de mexer’, traduz: **acoplamento**. Nem todo acoplamento é ruim. O ruim é o que **não se justifica**.”*

Os 5 tipos de acoplamento

Tipo	Definição	Exemplo (.NET)
Operacional	Um não roda sem o outro	<i>Pedidos trava se Pagamento fica lento</i>
Desenvolvimento	Não muda sem coordenar	<i>Mudar payload do Pagamento exige release coordenada</i>
Semântico	Compartilham conceitos implícitos	<i>Paid = 'capturado' (Pedidos) vs 'autorizado' (Pagamento)</i>
Funcional	Compartilham responsabilidade	<i>Finalizar pedido exige Pagamento + Estoque + Entrega</i>
Incidental	Quebram juntos sem motivo	<i>Pedidos importa lib interna do Pagamento para ganhar tempo</i>

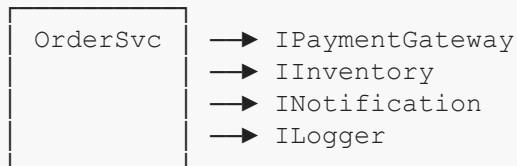
“Quando alguém diz ‘isso está acoplado’, a primeira pergunta é: acoplado **como?** Esses cinco tipos têm causas e remédios diferentes. Misturar é parte do problema.”

Aferente vs Eferente

Ca — Aferente: quantos componentes **dependem** de mim

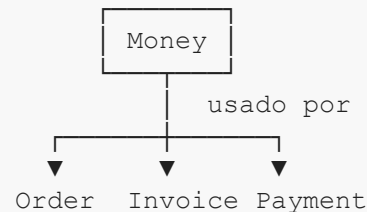
Ce — Eferente: de quantos componentes **eu dependo**

Alto Eferente (Ce ↑)



OrderService **depende de** 4 contratos externos

Alto Aferente (Ca ↑)



Money **é usado** pelo domínio inteiro

“OrderService depende de muita coisa — alto eferente. **Money** não depende de quase nada, mas o domínio inteiro depende dele. As duas situações pedem cuidados diferentes.”

Alto Eferente — OrderService

```
public class OrderService
{
    public OrderService(
        IPaymentGateway payment,
        IInventory inventory,
        INotification notification,
        ILogger<OrderService> logger)
    { /* ... */ }
}
```

Ce elevado: depende de 4 contratos externos

Alto Aferente — Money

```
public readonly record struct Money
(decimal Amount, string Currency)
{
    public Money Add(Money other)
    {
        if (Currency != other.Currency)
            throw new InvalidOperationException($"Cannot add
{Currency} and {other.Currency}");
        return new Money(Amount + other.Amount, Currency);
    }

    public Money Apply(decimal factor)
    {
        if (factor < 0)
            throw new
ArgumentOutOfRangeException(nameof(factor));
        return new Money(Amount * factor, Currency);
    }
}
```

Ca ↑ — guards explícitos = testes de unidade justificados

A consequência sobre testes

Componentes com alto eferente

- Testes de unidade viram pirâmide de mocks
- Cada dependência vira um mock
- Manutenção do teste > manutenção do código

Prefira testes de integração focados

Componentes com alto aferente

- Quebra aqui = quebra todo mundo que depende
- Mudança aqui se propaga
- Risco maior por mudança

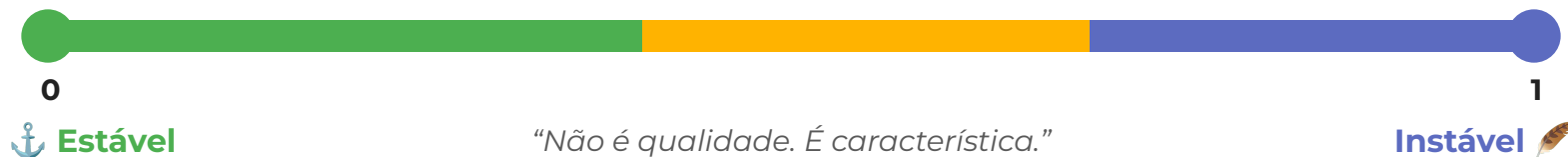
Exigem alta cobertura de teste

*“O tipo de acoplamento te diz **que tipo de teste vale a pena escrever**. Testar `OrderService` com 10 mocks não é qualidade, é teatro. Testar **Money** com cobertura quase total é proteção real.”*

Instabilidade: definição e fórmula

$$I = Ce \div (Ca + Ce)$$

I varia de 0 (estável) a 1 (instável)

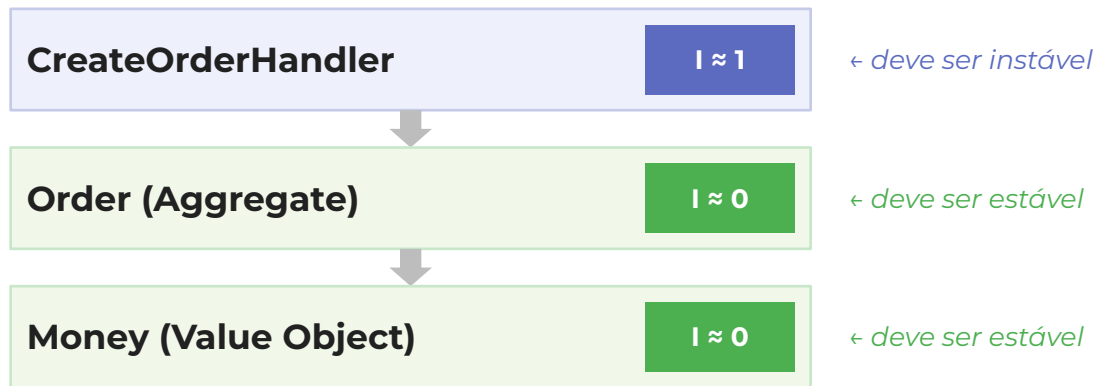


I próximo de 0
muita gente depende de mim
→ **Componente estável**

I próximo de 1
eu dependo de muita coisa
→ **Componente instável**

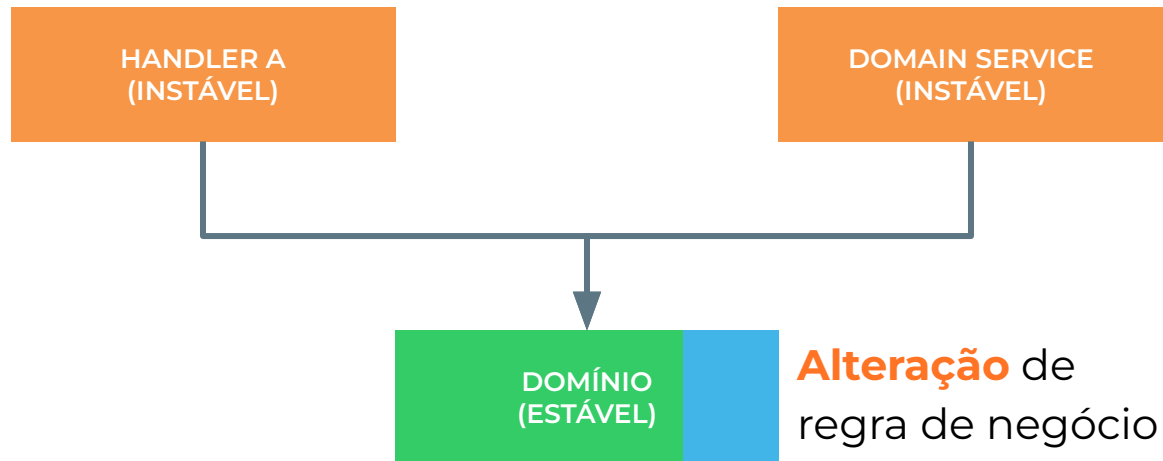
“Não decora a fórmula. Entende a leitura: quanto mais gente depende de você, mais **'pesado'** você fica. Quanto mais coisas você depende, mais **'leve'** você é.”

Estável vs Instável: o que isso significa



- Handler de feature **deve** ser instável: muda toda vez que a feature muda.
- Agregado **deve** ser estável: representa regra de negócio que não muda a cada release.
- Confundir os dois é a raiz de muito **sofrimento arquitetural**.

“Quando seu **agregado vira instável**, é porque ele está dependendo de coisas que mudam muito.
Quando seu handler vira estável, é porque ele virou central demais.”



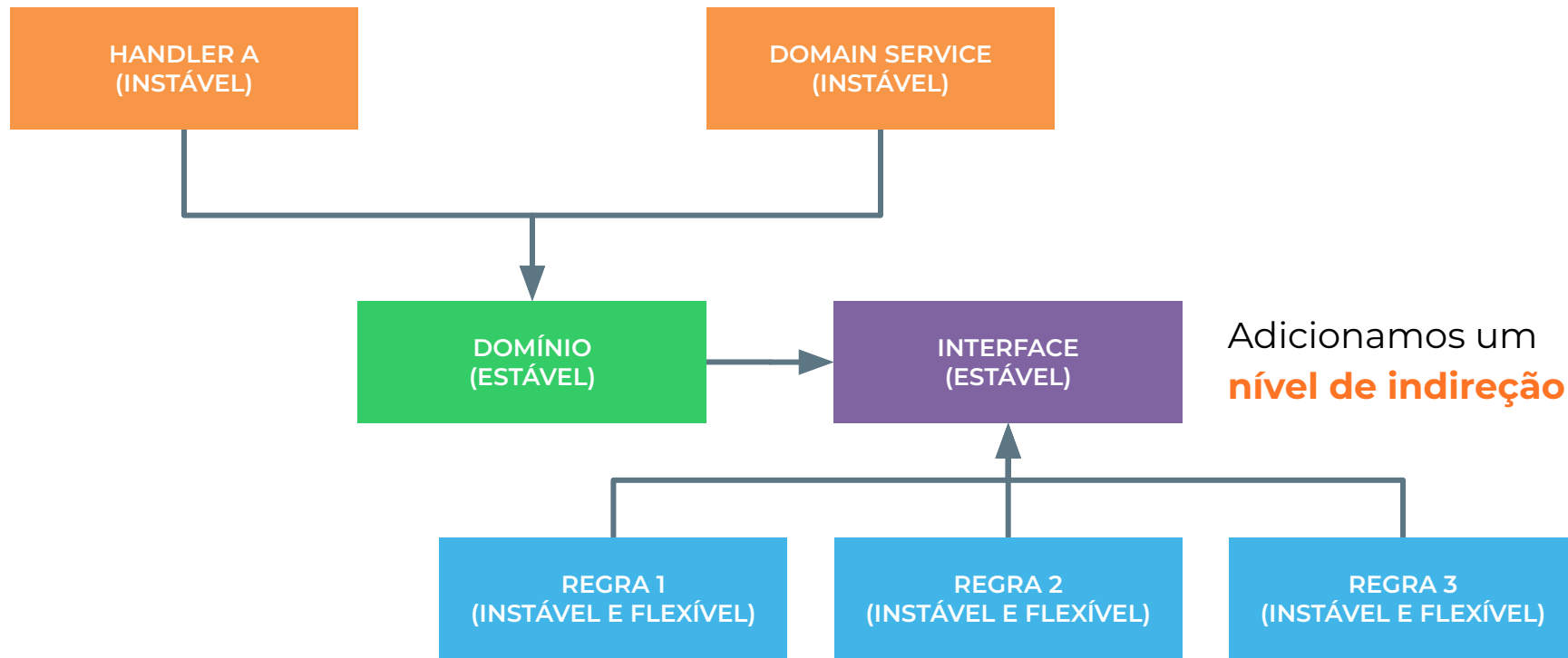
Componentes que são **desenhados para serem flexíveis devem ser fáceis de alterar, assim **eles devem ser instáveis**.**

Todos os problemas em ciência da computação podem ser resolvidos por outro nível de indireção, exceto o problema de muitos níveis de indireção.



*David J.
Wheeler*

Instabilidade e manutenabilidade



A armadilha dos níveis de indireção

*“Todo problema pode ser resolvido com mais um nível de indireção, exceto o problema dos **muitos níveis de indireção**.”*

— David J. Wheeler

Direto (1 passo)

```
public class CreateOrderHandler
{
    public async Task Handle(CreateOrderCommand cmd)
    {
        var order = new Order(cmd.CustomerId, cmd.Items);
        _db.Orders.Add(order);
        await _db.SaveChangesAsync();
    }
}
```

Indireto (4 passos)

```
public class CreateOrderHandler
{
    public async Task Handle(CreateOrderCommand cmd)
        => await _orderService.CreateOrder(cmd);
}

public class OrderService : IOrderService
{
    public Task CreateOrder(CreateOrderCommand cmd)
        => _factory.Create(cmd).Then(_repository.Save);
}

public class OrderFactory : IOrderFactory { /* ... */ }
public class OrderRepository : IOrderRepository { /* ... */ }
```

*“Indireção é remédio. Remédio sem dose vira veneno. Cada interface, cada camada, cada factory que você adiciona precisa estar **protegendo algo que muda**. Se não muda, é só ruído.”*

CAPÍTULO 2

ARQUITETURA E DESIGN

**COMPONENTES
RESPONSABILIDADES
RELACIONAMENTOS**



ESTRATÉGIA

(Objetivos de negócio,
restrições e atributos de
qualidade)

BUSCANDO O MENOR RISCO/CUSTO

Arquitetura sempre trata de



01 Objetivos do Negócio

02 Restrições

03 Atributos de Qualidade

04 Estratégia para Evolução

**Arquitetura é o
MAPA**



**Design é o
CAMINHO**



Para lembrar

**Arquitetura é
sempre sobre
Design!**

*Design nem sempre é
sobre Arquitetura*



**” Vamos integrar de forma
assíncrona (eventos) ou síncrona
(http) ”**

**” Vamos integrar de forma
assíncrona (eventos) ou síncrona
(http) ”**



Falamos sobre Arquitetura

**” No serviço X, vou usar retry +
backoff, idempotência e outbox
para publicar evento ”**

**” No serviço X, vou usar retry +
backoff, idempotência e outbox
para publicar evento ”**



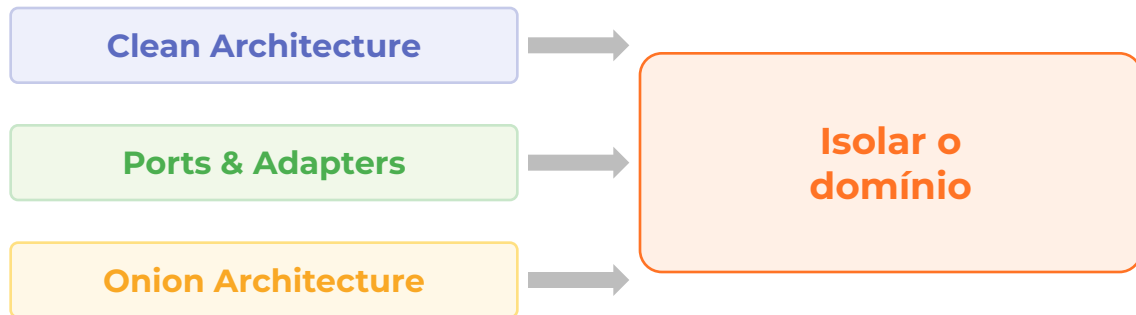
Falamos sobre Design

CAPÍTULO 3

CELAN E HEXAGONAL

Clean Architecture e Ports & Adapters são a mesma coisa.

Mesma intenção. Nomes diferentes. Onion idem.
O que muda é a implementação. E é aí que a maioria erra.



- Handler de feature **deve** ser instável: muda toda vez que a feature muda.
- Agregado **deve** ser estável: regra de negócio não muda a cada release.
- Confundir os dois é a raiz de muito **sofrimento arquitetural**.

*“Clean Architecture e Ports & Adapters são literalmente a mesma coisa. A diferença está nos diagramas e nos nomes. A **intenção é idêntica**.”*

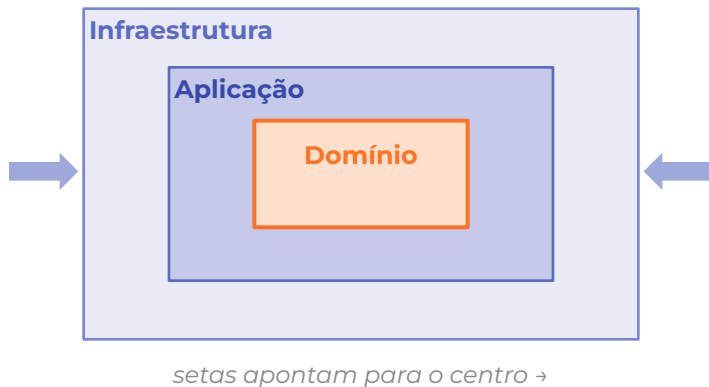
Alistair Cockburn (2005)
Hexagonal / Ports & Adapters
Problema: Testar lógica isolada de I/O
Resposta: Portas (interfaces) e adaptadores
Foco: O que entra e sai do sistema

Robert C. Martin (2012)
Clean Architecture
Problema: Regras de negócio sustentáveis
Resposta: Camadas concêntricas com Dependency Rule
Foco: O que protege o coração do sistema

Caminhos diferentes. Destino idêntico.

*“Cockburn estava lutando contra testes que dependiam do banco. Uncle Bob estava lutando contra código de negócio que apodreça. Os dois desenharam soluções com cara diferente, mas a mesma **topologia**.”*

“Depências apontam para dentro, em direção ao domínio.”

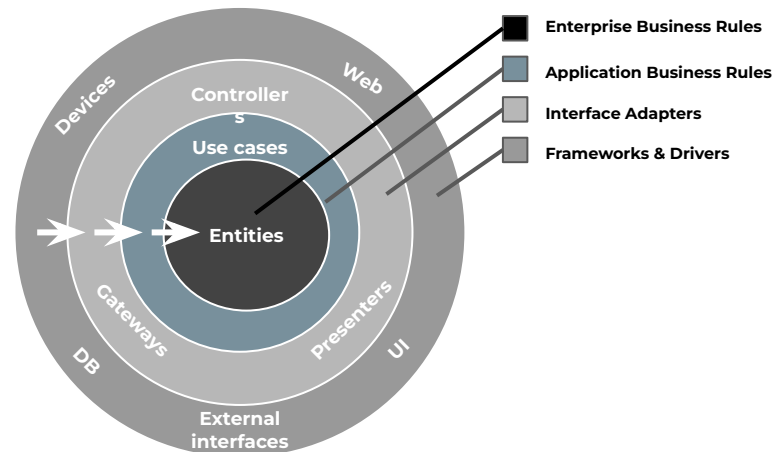
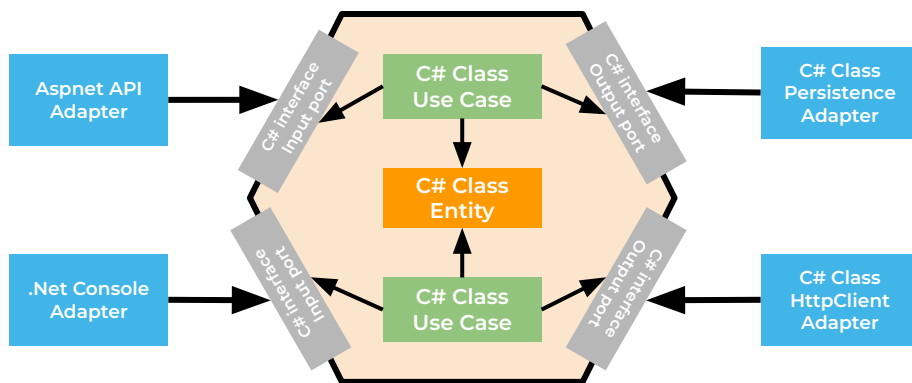


- Domínio não conhece infraestrutura
- Infraestrutura conhece domínio
- **Quando essa direção inverte**, a arquitetura adoeceu

“Se vocês só lembrarem de uma coisa desse bloco, lembrem dessa: **depências apontam para dentro**. Tudo o que Clean, Hexagonal e Onion propõem é como respeitar essa regra. Tudo.”

Hexagonal

Clean Architecture



“Mesma topologia. Desenhos diferentes.”

Cockburn: 'porta' = Uncle Bob: 'interface adapter' · 'adaptador' = 'frameworks' · Centro: sempre o domínio.

DEMO

Solution Explorer

```
MyApp.sln
|-- [proj] MyApp.Domain
|   |-- Entities/
|   |-- ValueObjects/
|   +-- Interfaces/
|-- [proj] MyApp.Application
|   |-- UseCases/
|   |-- DTOs/
|   +-- Interfaces/
|-- [proj] MyApp.Infrastructure
|   |-- Repositories/
|   |-- ExternalServices/
|   +-- Persistence/
+-- [proj] MyApp.API
    |-- Controllers/
    +-- Middlewares/
```

Pessoa abre o VS, clona um template 'Clean Architecture', já vem com 4 projetos. Força a aplicação a caber nessa estrutura. Os próximos slides são sobre os sintomas.

Quatro projetos criados antes mesmo de saber qual é o domínio.

MyApp.Domain

Entidades, regras de negócio

MyApp.Application

Casos de uso, orquestração

MyApp.Infrastructure

BD, APIs externas, cache

MyApp.API

Controllers, middleware

4 csproj. Zero linhas de domínio.

IUserService.cs

```
public interface IUserService
{
    Task<User> GetByIdAsync(
        Guid id);

    Task<User> CreateAsync(
        CreateUserDto dto);

    Task UpdateAsync(
        Guid id,
        UpdateUserDto dto);

    Task DeleteAsync(Guid id);
}
```

UserService.cs

```
public class UserService
    : IUserService
{
    public Task<User> GetByIdAsync(
        Guid id)
    { /* ... */ }

    public Task<User> CreateAsync(
        CreateUserDto dto)
    { /* ... */ }

    public Task UpdateAsync(
        Guid id, UpdateUserDto dto)
    { /* ... */ }

    public Task DeleteAsync(
        Guid id) { /* ... */ }
}
```

Diagnóstico: dois arquivos para uma ideia. A interface não protege de nada — só duplica a assinatura para “permitir mockar”.

Quantas interfaces no projeto têm exatamente uma implementação com o mesmo nome menos o 'I'? Cada uma é um arquivo a mais para manter, sem ganho real.

IRepository.cs

O que se vê em todo projeto:

```
public interface IRepository<T>
    where T : class
{
    Task<T?> GetByIdAsync(object id);
    Task<IEnumerable<T>> GetAllAsync();
    Task<IEnumerable<T>> FindAsync(
        Expression<Func<T,bool>> pred);
    Task AddAsync(T entity);
    void Update(T entity);
    void Remove(T entity);
}
```



AppDbContext.cs

O que o EF Core já oferece:

```
public class AppDbContext : DbContext
{
    public DbSet<User> Users { get;set;}

    // GetByIdAsync
    // -> Users.FindAsync(id)
    // GetAllAsync
    // -> Users.ToListAsync()
    // FindAsync
    // -> Users.Where(p).ToListAsync()
    // AddAsync
    // -> Users.AddAsync(entity)
    // Update / Remove
    // -> Users.Update/Remove(entity)
}
```

Diagnóstico: a abstração não esconde nada. Reimplementa o DbSet<T> com nomes diferentes e impede recursos do EF (Include, AsNoTracking, projeções) sem furar a abstração.

'Estou abstraindo o EF para poder trocar.' Trocar pelo quê? E quando? E mesmo que troque, IRepository não ajuda — cada query do EF tem comportamento que essa interface não modela.

CreateOrderUseCase.cs

Application

```
public class CreateOrderUseCase
    : ICreateOrderUseCase
{
    private readonly IOrderService
        _service;

    public CreateOrderUseCase(
        IOrderService service)
        => _service = service;

    public Task<OrderDto> Execute(
        CreateOrderInput input)
        => _service.CreateOrder(
            input);
}
```

OrderService.cs

Application

```
public class OrderService
    : IOrderService
{
    private readonly
        IOrderRepository _repo;

    public OrderService(
        IOrderRepository repository)
        => _repo = repository;

    public Task<OrderDto>
        CreateOrder(input)
        => _repo.Create(
            input.ToEntity())
            .ContinueWith(...);
}
```

OrderRepository.cs

Infrastructure

```
public class OrderRepository
    : IOrderRepository
{
    private readonly AppDbContext
        _context;

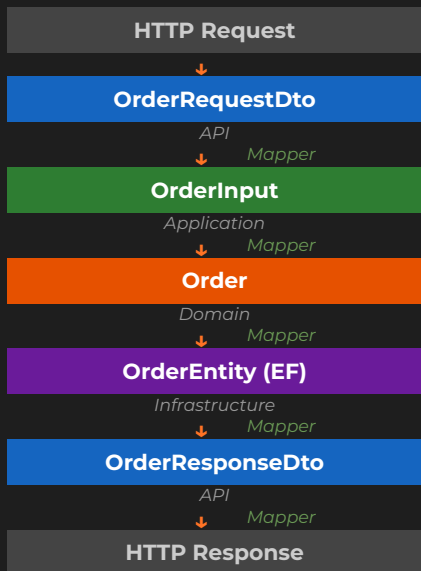
    public OrderRepository(
        AppDbContext context)
        => _context = context;

    public async Task<Order>
        Create(Order order) {
        _context.Orders.Add(order);
        await _context
            .SaveChangesAsync();
    }
}
```

Diagnóstico: três arquivos, três interfaces, zero lógica adicionada. Cada camada existe para “respeitar a arquitetura”. Esse padrão é chamado **passthrough** — e é dívida pura.

UseCase chama Service. Service chama Repository. Nenhum dos dois primeiros adiciona regra, validação ou transformação relevante. Estão lá para preencher a estrutura. Cada interface dessas é mock no teste.

Fluxo de transformação



Models.cs — a mesma informação, 5 representações

```
// API
public record OrderRequestDto(
    Guid CustomerId, List<ItemDto> Items);

// Application
public record OrderInput(
    Guid CustomerId, List<ItemInput> Items);

// Domain
public class Order { public Guid CustomerId;
    public List<OrderItem> Items; }

// Infrastructure
public class OrderEntity { public Guid CustomerId;
    public List<OrderItemEntity> Items; }

// API (response)
public record OrderResponseDto(
    Guid Id, Guid CustomerId,
    List<ItemDto> Items, decimal Total);
```

Diagnóstico: a mesma informação atravessa quatro representações. Cada mapper é código. Cada divergência é bug potencial. Um campo novo: 5 arquivos para tocar.

Quer adicionar um campo? Dto da API, Input da Application, entidade de Domínio, Entity do EF, DtoResponse e todos os mappers. Em projeto grande, isso é o que segura release.

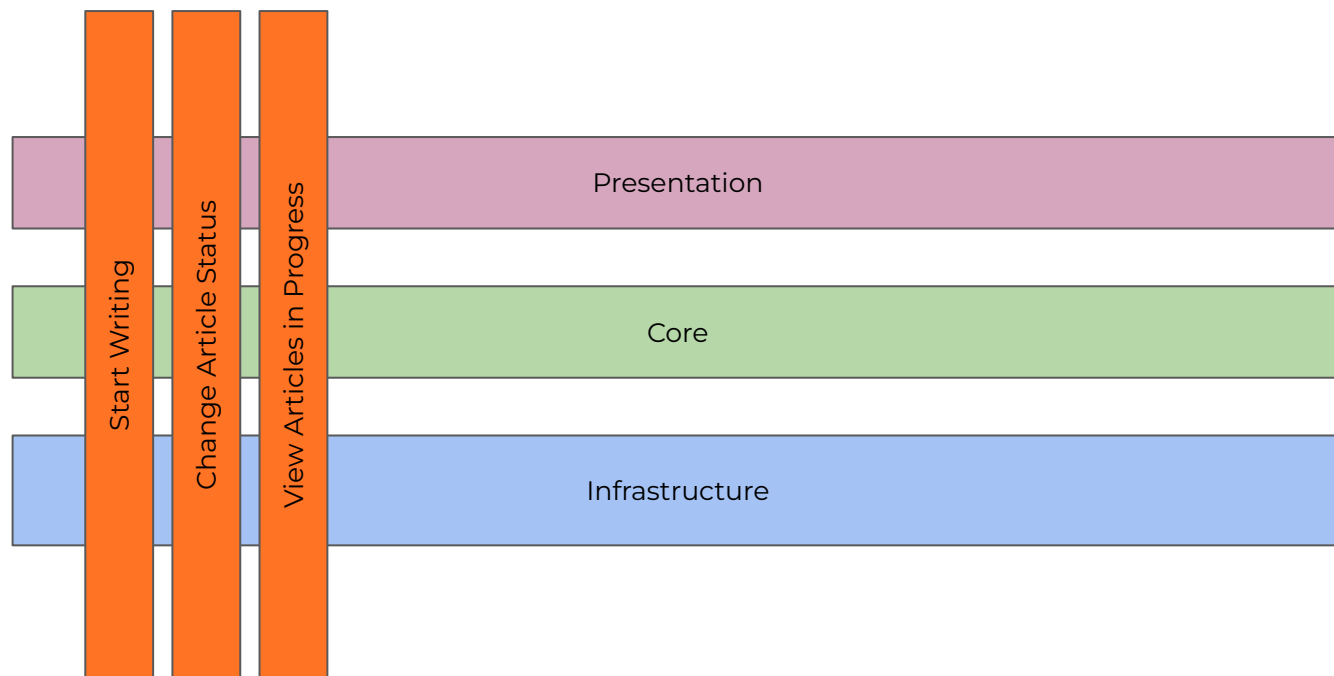
Não é opinião. É diagnóstico.

O que vimos no código	Nome técnico
Mudança simples toca vários projetos	Acoplamento de desenvolvimento alto e desnecessário
Handler/Service muda toda vez que agregados de domínio mudam	Acoplamento incidental entre camadas
Domínio “puro” depende de cascata de modelos para chegar ao banco	Instabilidade mal distribuída — o que devia ser estável virou frágil
Teste de UseCase/Handler precisa mockar Service, que mocka Repository	Alto acoplamento eferente testado por unidade — pirâmide de mocks

“Os sintomas têm nome”

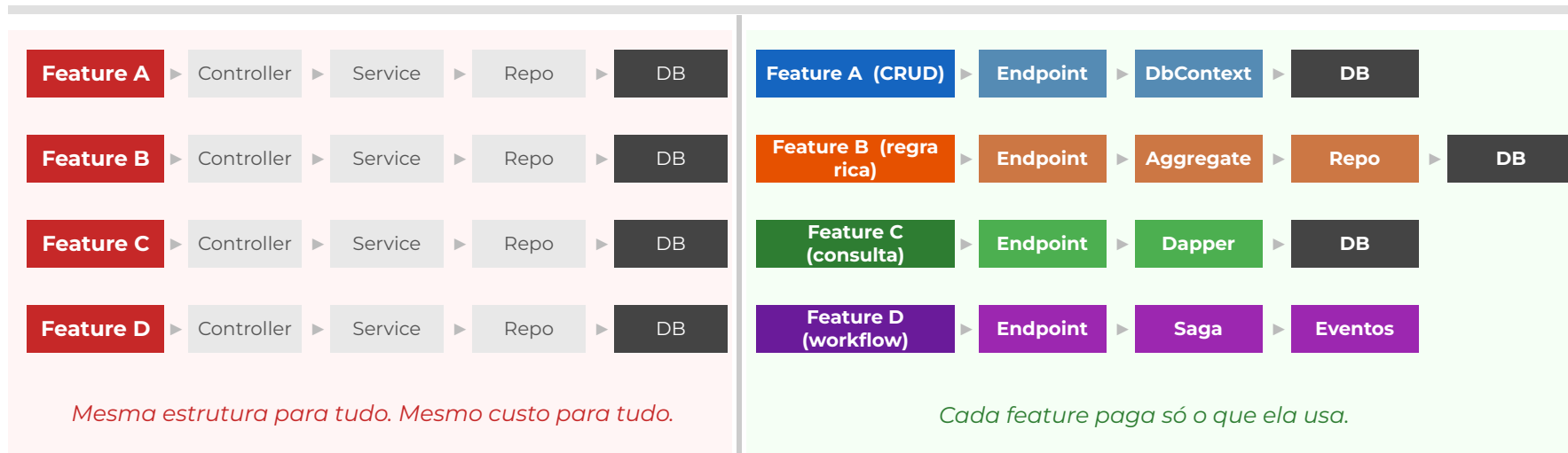
CAPÍTULO 4

VERTICAL SLICE ARCHITECTURE



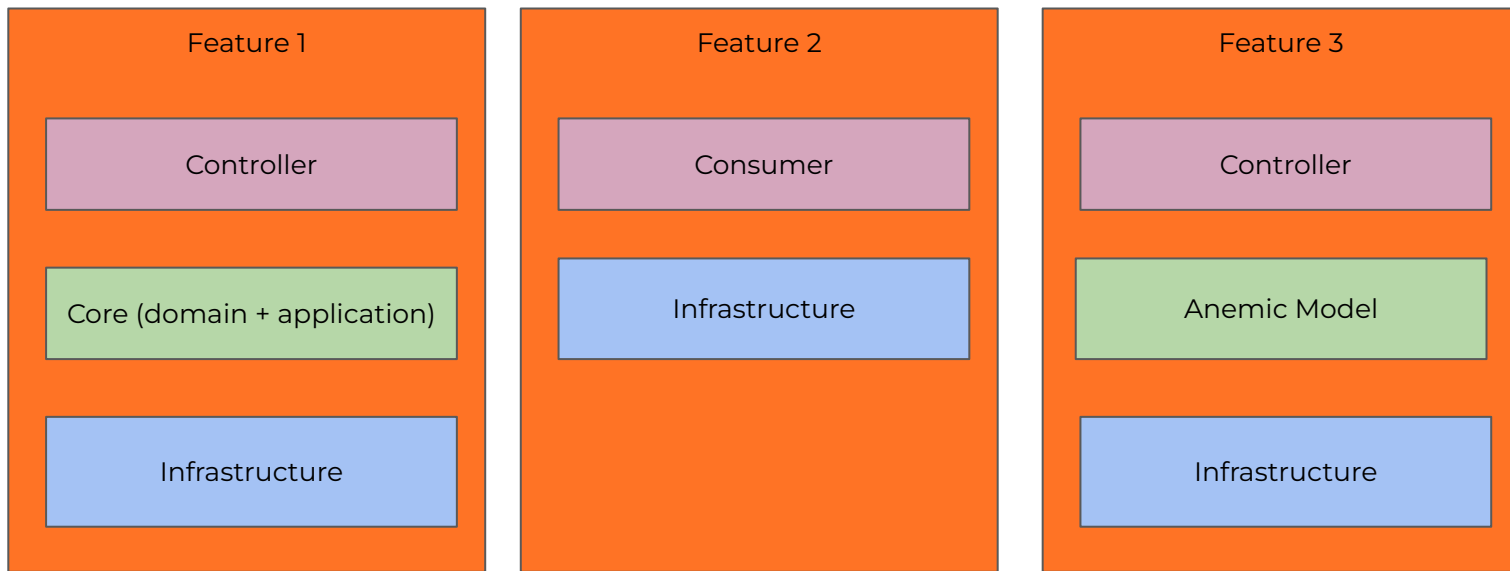
Arquitetura não precisa ser uniforme.

Diferentes features têm diferentes necessidades.



Forçar todas pelo mesmo esqueleto é cobrar imposto onde não há renda.

Cada slice é autocontido. O que muda junto, fica junto

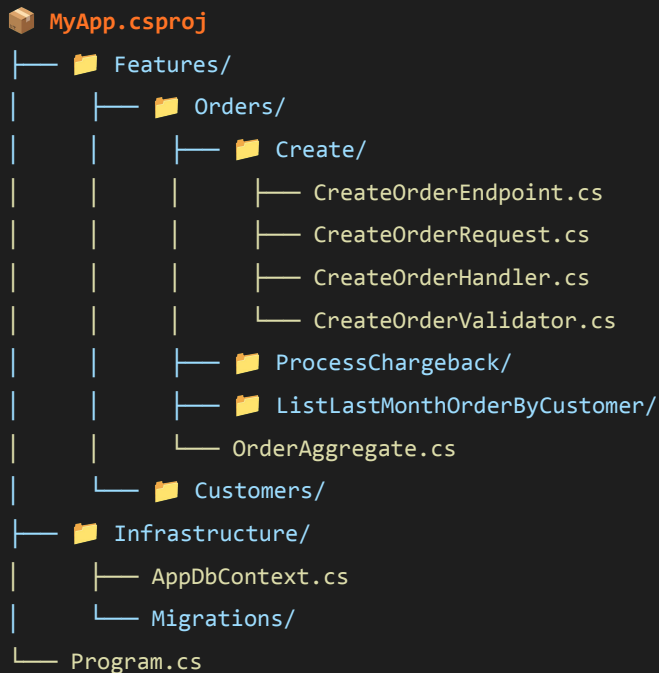


*Quando precisam mudar uma feature, qual a chance de mudar Controller, Service, Repo e Entity dela ao mesmo tempo?
Altíssima.*

A feature ao lado? Baixíssima. A organização precisa refletir essa realidade.

Como isso fica em C#

Solution Explorer



```
MyApp.csproj
├── Features/
│   ├── Orders/
│   │   └── Create/
│   │       ├── CreateOrderEndpoint.cs
│   │       ├── CreateOrderRequest.cs
│   │       ├── CreateOrderHandler.cs
│   │       └── CreateOrderValidator.cs
│   ├── ProcessChargeback/
│   ├── ListLastMonthOrderByCustomer/
│   └── OrderAggregate.cs
├── Customers/
├── Infrastructure/
│   └── AppDbContext.cs
├── Migrations/
└── Program.cs
```

Por que isso funciona?

Uma feature = uma pasta

Tudo que serve Orders/Create está em Orders/Create. Zero busca cross-feature.

Um único .csproj

Sem projetos separados por camada. Sem referências circulares. Sem cerimônia de setup.

Nenhuma dependência nova

É organização de pastas. Pode ser feito hoje, em qualquer projeto .NET.

Testabilidade imediata

Cada handler é uma classe simples. Unit test = `new Handler().Handle(cmd)`.

Um único .csproj. Cada feature numa pasta. Cada arquivo na feature que ele serve.

REPR Pattern (Request - Endpoint - Response)



```
public class MyEndpoint : Endpoint<MyRequest, MyResponse>
{
    public override void Configure()
    {
        Post("/hello/world");
        AllowAnonymous();
    }

    public override async Task HandleAsync(MyRequest r, CancellationToken c)
    {
        var response = MyEndpointHandler.Handle(r);
        await SendAsync(r);
    }
}
```

DEMO

CAPÍTULO 5

INSIGHTS & TAKEWAYS

Insights & Takeaways

Obsessão por eliminar acoplamento

Uso indiscriminado de interfaces para favorecer o "baixo acoplamento"

Complexidade accidental

Para alcançar um "baixo acoplamento", eventualmente pagamos com o aumento da complexidade do projeto

Insights & Takeaways

Proteger o que é volátil

Devemos orientar a arquitetura e design para
"proteger" as partes mais voláteis

Cuidado com integrações entre sistemas

Um vilão, muitas vezes, invisível são as integrações
entre serviços, então trate elas com carinho

CLUBE DE ESTUDOS
BY ELEMAR JR 